SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1153

COURSE OUTCOME COVERED

CO4:
implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS *(BL5)*

Assessment Tool Weightage: 40%

QUESTIONS: 2 **Total Marks:10**

Annexure A

Industry Problem Solving Task

Code of Conduct:

*I **Annajothi S** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: *Annajothi S*

Annexure A

Industry Problem Solving Task

2.A **hospital management system** is designed to manage patient records, appointments, billing, and doctor schedules. The system is structured using **object-oriented layered architecture**, where the **view layer** provides interfaces for staff and patients, the **business logic layer** handles appointment scheduling and billing rules, and the **access layer** manages interactions with an OODBMS.

The developers use **Factory pattern** to create different user objects (doctor, patient, admin) and **Observer pattern** to notify patients about appointment updates. The system ensures modularity, allowing new modules such as telemedicine to be added easily.

Question:

Evaluate how object-oriented design principles and patterns are used in this system, and explain how the layered architecture supports scalability and efficient integration with the database.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Hospital Management System Using Object-Oriented Layered Architecture

Introduction

A **Hospital Management System (HMS)** is a software application designed to automate and manage the daily operations of a hospital. It helps healthcare providers organize patient information, schedule appointments, manage doctor availability, generate bills, and maintain medical records efficiently. By using an **Object-Oriented Layered Architecture**, the system becomes modular, reusable, scalable, and easier to maintain. In this architecture, responsibilities are divided into different layers, ensuring that changes in one part of the system have minimal impact on other parts.

1. Object-Oriented Layered Architecture

The Hospital Management System is divided into three major layers:

1. **View Layer (Presentation Layer)**
2. **Business Logic Layer**
3. **Access Layer (Data Access Layer)**

Each layer performs a specific role and communicates with the adjacent layers.

A. View Layer (Presentation Layer)

The View Layer is responsible for interacting with users. It provides graphical interfaces or web pages through which patients, doctors, administrators, and hospital staff can use the system.

Responsibilities

- Display login and registration pages.
- Allow patients to book or cancel appointments.
- Enable doctors to view schedules and patient records.
- Allow administrators to manage users and hospital operations.
- Display billing information and payment status.
- Show notifications related to appointments.

Example

When a patient logs in, they can:

- View available doctors.
- Book an appointment.
- Check previous medical records.
- Receive appointment confirmations.

The View Layer does not perform calculations or database operations directly; instead, it forwards requests to the Business Logic Layer.

B. Business Logic Layer

The Business Logic Layer acts as the core of the system and contains all business rules and decision-making processes.

Responsibilities

- Validate patient and doctor information.
- Schedule appointments without conflicts.
- Manage doctor availability.
- Generate hospital bills based on services used.
- Process appointment cancellations or rescheduling.
- Calculate discounts, insurance claims, or taxes.
- Trigger notifications when appointment details change.

Example

Suppose a patient requests an appointment with a doctor at 10:00 AM:

1. The Business Logic Layer checks whether the doctor is available.
2. If the slot is free, it confirms the booking.
3. If the slot is occupied, it suggests another available time.
4. The appointment information is then stored through the Access Layer.

This layer ensures that hospital policies and operational rules are consistently followed.

C. Access Layer (Data Access Layer)

The Access Layer communicates with the **Object-Oriented Database Management System (OODBMS)**. It handles all interactions with the database while hiding implementation details from other layers.

Responsibilities

- Store patient records.
- Retrieve doctor schedules.
- Save appointment details.
- Update billing information.
- Delete or modify records when necessary.

Benefits

- Isolates database operations from business logic.
- Simplifies future database changes.
- Improves security and maintainability.
- Supports efficient data retrieval and storage.

2. Use of Object-Oriented Principles

The system uses object-oriented concepts such as:

Encapsulation

Each class stores its own data and methods. For example, a Patient object contains personal details and methods to update or retrieve information.

Inheritance

Different user types such as Doctor, Patient, and Admin can inherit common properties from a base User class.

Polymorphism

The same interface can perform different actions depending on the object type. For example, different users may implement their own version of a login or dashboard display.

Abstraction

Complex implementation details, such as database access or billing calculations, are hidden from end users.

3. Factory Pattern

The **Factory Design Pattern** is used to create user objects dynamically without exposing object creation logic.

Purpose

Instead of creating objects directly using constructors, a factory method decides which type of object to instantiate.

Example

```
User user = UserFactory.createUser("Doctor");
```

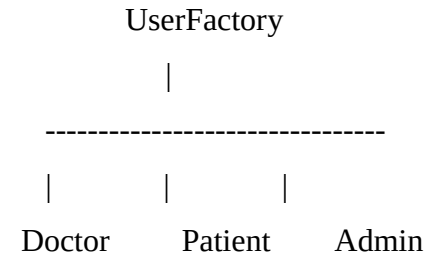
Depending on the input, the factory creates:

- Doctor object
- Patient object
- Admin object

Advantages

- Reduces code duplication.
- Centralizes object creation.
- Makes it easy to introduce new user roles, such as Nurse or Pharmacist.
- Improves maintainability and scalability.

Simple Representation



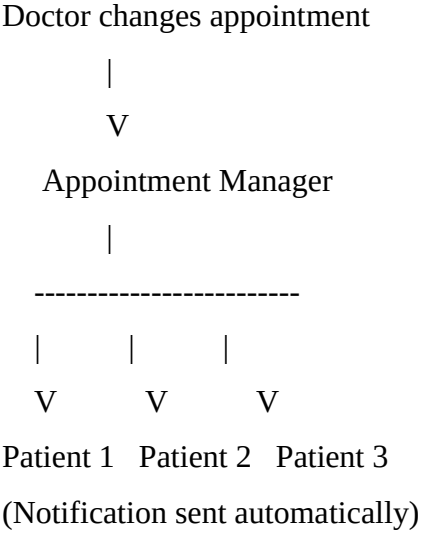
4. Observer Pattern

The **Observer Design Pattern** enables automatic notifications whenever appointment information changes.

How It Works

- The Appointment System acts as the **Subject**.
- Patients act as **Observers**.
- Whenever an appointment is booked, cancelled, or rescheduled, all registered observers receive notifications automatically.

Example Workflow



Notifications Can Include

- Appointment confirmation.
- Appointment reminder.
- Cancellation notice.

- Rescheduled appointment details.

Advantages

- Loose coupling between components.
- Real-time updates.
- Improved patient communication.
- Easier addition of new notification channels such as SMS, email, or mobile app alerts.

5. Modularity and Extensibility

Because the system follows layered architecture and object-oriented principles, new modules can be added with minimal impact on existing functionality.

Example: Telemedicine Module

A Telemedicine Module can provide:

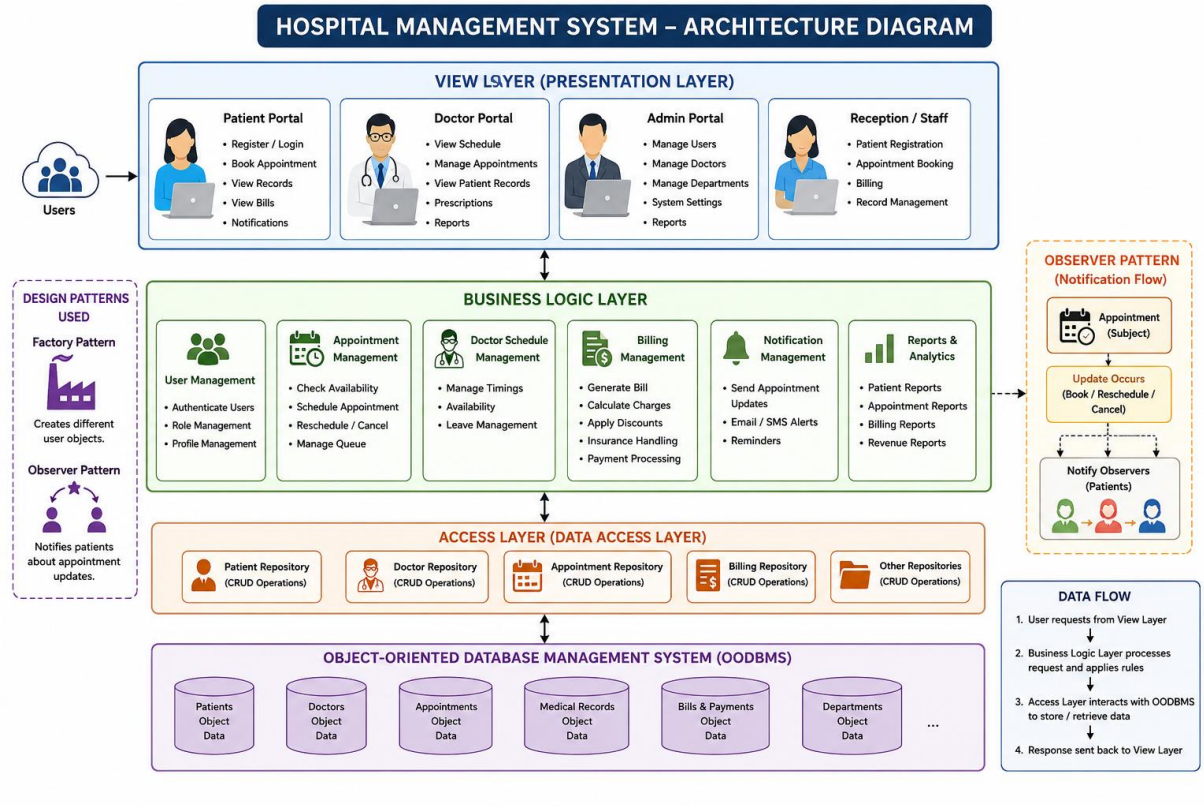
- Online doctor consultations.
- Video appointments.
- Digital prescriptions.
- Remote patient monitoring.

Since responsibilities are separated into layers, this module can reuse existing authentication, appointment scheduling, and billing services while introducing its own specialized features.

Other modules that can be integrated similarly include:

- Pharmacy Management
- Laboratory Management
- Inventory Management
- Ambulance Services
- Insurance Processing
- Electronic Health Records (EHR)

6.Overall System Flow



7. Advantages of This Architecture

- **Separation of concerns:** Each layer has a clearly defined responsibility.
- **Scalability:** New features and modules can be added without major redesign.
- **Maintainability:** Bugs and updates are easier to manage because functionality is organized into layers.
- **Reusability:** Common components and classes can be reused across modules.
- **Flexibility:** Design patterns such as Factory and Observer allow the system to adapt to new requirements.
- **Security:** Direct database access is restricted to the Access Layer.
- **Testability:** Individual layers can be tested independently.

8. Conclusion

The Hospital Management System is effectively designed using an **Object-Oriented Layered Architecture**, separating the presentation, business logic, and data access responsibilities into distinct layers. This organization improves code quality, maintainability, and scalability. The **Factory Pattern** simplifies the creation of different user objects such as

doctors, patients, and administrators, while the **Observer Pattern** ensures that patients receive timely notifications about appointment updates. The modular architecture also supports future enhancements, such as telemedicine and online consultation services, making the system adaptable to evolving healthcare needs. This design provides an efficient, reliable, and extensible solution for managing modern hospital operations.